

Masked Autoencoder implémenté à la main : pré-entraînement auto-supervisé et classification de défauts de surface

Philemon Vittet

Cours d'introduction à la computer vision

Code : <https://github.com/PhilVitt/mae-from-scratch>

Résumé

Ce rapport décrit la réimplémentation complète d'un Masked Autoencoder (MAE) sans différentiation automatique ni couches préfabriquées. Le passage avant et la rétropropagation de chaque couche, attention et normalisation comprises, sont dérivés et codés à la main, puis validés par différences finies (erreur relative inférieure à 10^{-6}). torch n'est utilisé que comme bibliothèque de tableaux, ce qui permet d'exécuter ces opérations manuelles sur GPU (Apple MPS) et donc d'entraîner dans des temps raisonnables. Le modèle est étudié sur STL-10, où l'ablation du taux de masquage illustre l'étude centrale de l'article d'origine, puis appliqué à la classification de six défauts de surface d'acier (jeu de données NEU). Pré-entraîné sans étiquettes, son encodeur gelé alimente une sonde linéaire qui atteint 90,6% d'exactitude, contre 78,9% pour un encodeur non entraîné; l'écart se creuse quand peu d'images sont étiquetées.

1 Introduction

Le coût de l'annotation est un frein récurrent en vision industrielle : étiqueter des défauts demande un opérateur expert, et chaque ligne de production change la distribution des images. L'apprentissage auto-supervisé permet de réduire ce coût, en apprenant des représentations sur des images non étiquetées avant de n'utiliser les étiquettes que pour une tâche aval légère.

Le Masked Autoencoder [1] masque une grande partie des patches d'une image et entraîne un Transformer à reconstruire les pixels manquants. Ce travail vise trois objectifs. Comprendre le MAE en le réimplémentant intégralement, ce qui impose de dériver à la main la rétropropagation de l'attention et de la normalisation, plutôt que de s'appuyer sur l'autograd d'un framework. Vérifier ensuite, sur STL-10, que le pré-entraînement apprend des représentations utiles, mesurées par sonde linéaire, et observer l'effet du taux de masquage. Appliquer enfin la méthode à un jeu de données industriel de défauts d'acier et évaluer son intérêt quand les étiquettes sont rares.

2 Travaux liés

Le MAE prolonge les autoencodeurs débruiteurs [6] et transpose à l'image la modélisation masquée de BERT [4], où seules les positions masquées contribuent à la perte. Son ossature est le Vision Transformer [2], qui découpe l'image en patches et applique un Transformer [3]. BEiT [5] prédisait des tokens discrets ; le MAE montre qu'il suffit de reconstruire les pixels. La détection de défauts de surface repose aujourd'hui sur des réseaux convolutifs et des détecteurs profonds [11, 12] ; le jeu de données NEU [9, 10]

en est une référence. Ces méthodes sont supervisées, là où le pré-entraînement auto-supervisé étudié ici cherche à réduire le besoin d'étiquettes.

3 Méthode

Tokenisation. Une image 96×96 est découpée en patches 8×8 , soit une grille $12 \times 12 = 144$ patches aplatis en vecteurs de dimension $8 \cdot 8 \cdot 3 = 192$, projetés linéairement vers une dimension d'embedding D . On ajoute un encodage positionnel sinusoïdal 2D fixe : l'auto-attention est invariante à l'ordre, sans position le modèle ne sait pas où se situe chaque patch. Le choix de patches 8×8 plutôt que 16×16 divise par deux la taille du bloc reconstruit et réduit nettement l'aspect en damier des reconstructions, au prix d'une séquence quatre fois plus longue.

Masquage. Pour chaque image, un bruit aléatoire sur les 144 positions est trié, ce qui fournit une permutation. Les $N_{vis} = \lfloor 144(1-r) \rfloor$ premiers patches sont conservés et les autres masqués, r étant le taux de masquage. Les indices de restauration sont mémorisés pour replacer la séquence dans son ordre d'origine côté décodeur ; la correction du masquage dépend de la cohérence entre cette permutation et son inverse.

Encodeur et décodeur asymétriques. L'encodeur n'applique ses blocs Transformer qu'aux patches visibles : à 75 % de masquage, il ne traite que 36 tokens sur 144, et les tokens masqués n'y entrent jamais. Le décodeur projette les tokens encodés vers une dimension plus étroite, insère un token masqué appris (un vecteur partagé) à chaque position masquée, restaure l'ordre d'origine, ajoute son encodage positionnel, applique quelques blocs et prédit les pixels de chaque patch par une tête linéaire. Plus petit que l'encodeur, il n'est utilisé que pour le pré-entraînement.

Perte. La cible est l'image découpée en patches, chaque patch étant normalisé par sa moyenne et son écart-type. La perte est l'erreur quadratique moyenne sur les seuls patches masqués :

$$\mathcal{L} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \frac{1}{192} \|\hat{x}_i - \tilde{x}_i\|^2. \quad (1)$$

Sonde linéaire. La qualité des représentations est mesurée en gelant l'encodeur, en le passant sur les images étiquetées sans masquage, en moyennant les tokens (moyennage global) en un vecteur par image, puis en entraînant un unique classifieur linéaire par régression logistique [1].

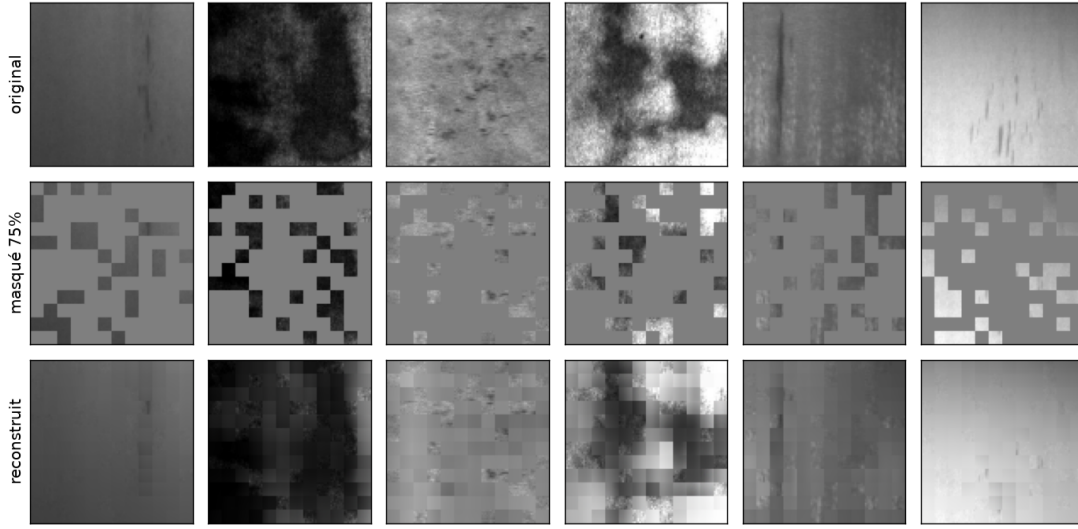


FIGURE 1 – Le MAE appliqué aux défauts de surface d’acier. À partir des 25 % de patches visibles (ligne du milieu), le décodeur reconstruit l’image (ligne du bas) ; l’encodeur ainsi pré-entraîné, sans étiquettes, sert ensuite à classer le type de défaut.

La référence est le même protocole sur un encodeur non entraîné.

4 Implémentation

Tout à la main, sur GPU. Aucune fonction de différenciation automatique ni de `torch.nn` n’est utilisée : `torch` ne sert que de bibliothèque de tableaux. Chaque couche est une classe qui implémente son passage avant *et* sa rétropropagation, sur le modèle d’une couche linéaire qui met en cache son entrée. Les paramètres sont des attributs (`W`, `gamma`...) et leurs gradients portent le nom préfixé (`dW`, `dgamma`...), ce qui permet à l’optimiseur de parcourir le modèle uniformément. Les couches écrites sont : linéaire, LayerNorm, GELU, attention multi-têtes, MLP, bloc Transformer pré-norm, encodeur, décodeur et perte. Les deux dérivées non triviales sont celles de la normalisation (listing 1) et de l’attention, dont le point délicat est la jacobienne du softmax, les logits ayant été mis à l’échelle par $1/\sqrt{d_k}$.

Listing 1 – Rétropropagation manuelle de LayerNorm.

```
def backward(self, dy):
    xhat, std = self.cache
    D = xhat.shape[-1]
    self.dgamma = (dy*xhat).reshape(-1,D).sum(0)
    self.dbeta = dy.reshape(-1,D).sum(0)
    dxhat = dy * self.gamma
    dx = (dxhat - dxhat.mean(-1,keepdim=True)
          - xhat*(dxhat*xhat).mean(-1,keepdim=True)
          ) / std
    return dx
```

Validation des gradients. Chaque rétropropagation est vérifiée par différences finies centrées, sur CPU en double précision : un paramètre est perturbé de $\pm \epsilon$, la variation de la perte est comparée au gradient analytique. Sur le MAE complet, la pire erreur relative est inférieure à 10^{-6} . Ce

embedding D	128
blocs encodeur / décodeur	6 / 2
têtes	4
dimension décodeur	64
ratio MLP	4
patch / résolution	8 / 96×96
taux de masquage	0,75
optimiseur	AdamW, $\beta = (0,9, 0,95)$
taux d’apprentissage	$8 \cdot 10^{-4}$
weight decay	0,05
taille de lot	256
épouques	100 (STL) / 600 (NEU)

TABLE 1 – Hyperparamètres.

contrôle est nécessaire : un gradient erroné ne provoque pas d’erreur d’exécution, il produit seulement un modèle qui n’apprend pas. Le sur-apprentissage d’un seul batch, masqué figé, sert de second test : la perte y chute, ce qui confirme que le graphe de gradients est correct de bout en bout.

CPU pour vérifier, GPU pour entraîner. Le même code tourne sur deux configurations selon l’objectif. Le gradient check exige de la précision : il s’exécute sur CPU en `float64`. L’entraînement exige de la vitesse : il s’exécute sur le GPU Apple (MPS) en `float32`, où les opérations matricielles écrites à la main sont environ huit fois plus rapides que sur CPU. C’est ce qui rend abordables les patches 8×8 et un modèle de quelques millions de paramètres.

Configuration. Le modèle reste modeste (environ 1,3 million de paramètres) ; les hyperparamètres sont donnés table 1. L’optimiseur est Adam [7] avec weight decay dé-couplé (AdamW [8]) sur les seules matrices de poids ; le taux d’apprentissage suit une montée linéaire puis une décroissance en cosinus.

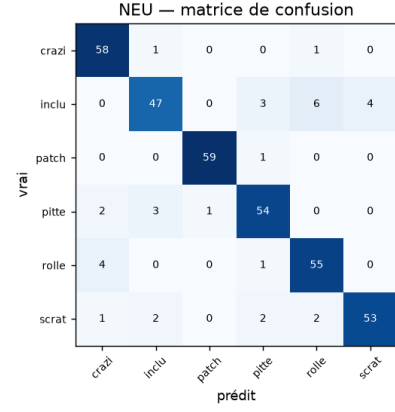
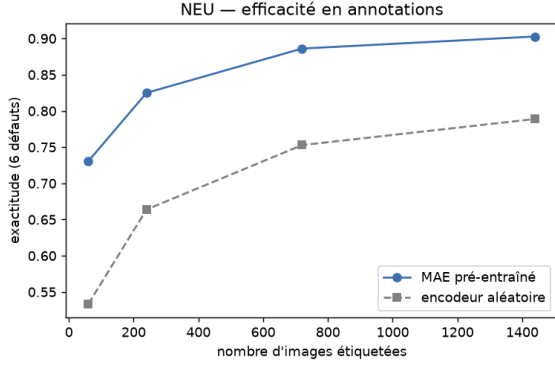


FIGURE 2 – NEU. Gauche : exactitude de la sonde selon le nombre d’images étiquetées, MAE contre encodeur aléatoire. Droite : matrice de confusion du MAE sur les six défauts.

	aléatoire	MAE
STL-10, sonde linéaire (10 classes)	42,2 %	53,3 %
NEU, sonde linéaire (6 défauts)	78,9 %	90,6 %
NEU, 10 étiquettes par classe	53,3 %	73,1 %

TABLE 2 – Exactitude de la sonde linéaire, encodeur aléatoire contre MAE pré-entraîné.

5 Protocole expérimental

STL-10 (96×96 , 10 classes) sert d’étude de méthodologie : pré-entraînement sur 12000 images non étiquetées, sonde linéaire sur 5000 images étiquetées, évaluation sur 8000 images de test. NEU-CLS sert de cas applicatif : 1800 images de surface d’acier en six défauts (crazing, inclusion, patches, pitted surface, rolled-in scale, scratches), en niveaux de gris, dupliquées sur trois canaux pour réutiliser la même configuration, redimensionnées en 96×96 et réparties de façon stratifiée en 1440 images d’entraînement et 360 de test. Le MAE est pré-entraîné sans étiquettes, puis la sonde linéaire est entraînée et évaluée ; le nombre d’images étiquetées fournies à la sonde est ensuite varié pour mesurer l’apport du pré-entraînement à faible budget d’annotation.

6 Résultats

6.1 NEU : classification de défauts

Pré-entraîné sans étiquettes sur les images d’acier, le MAE fait décroître la perte de reconstruction de 3,03 à 0,85 et reconstruit la texture des défauts à partir de 25 % des patches (figure 1). La sonde linéaire posée sur l’encodeur gelé atteint 90,6 % d’exactitude sur les six défauts, contre 78,9 % pour un encodeur aléatoire (table 2). L’apport du pré-entraînement est surtout net à faible budget d’annotation : l’écart se creuse quand peu d’images sont étiquetées, par exemple 73,1 % contre 53,3 % avec dix images par classe (figure 2, gauche). La matrice de confusion (figure 2, droite) montre que les erreurs résiduelles concernent surtout les défauts visuellement proches.

6.2 STL-10 : ablation du taux de masquage

Sur STL-10, la reconstruction porte sur des objets variés (figure 3). Le pré-entraînement fait décroître la perte de reconstruction (figure 4), et la sonde linéaire dépasse l’encodeur aléatoire, de 42,2 % à 53,3 %, ce qui indique que des représentations utiles ont été apprises. La figure 5 reporte l’exactitude selon le taux de masquage, contre l’encodeur aléatoire et le hasard (10 % pour 10 classes). Les quatre taux dépassent tous l’encodeur aléatoire d’environ dix points, donc le gain tient surtout au pré-entraînement, pas au réglage fin du masquage. Le maximum est à 0,75 (53,3 %), le taux retenu par l’article [1], mais l’écart entre taux reste faible (51,6 à 53,3 %). À cette échelle, la quantité de données et la taille du modèle pèsent plus que le taux de masquage. La perte de reconstruction décroît quand on masque moins (0,47 à 0,25 contre 0,71 à 0,9), ce qui est attendu puisque la tâche devient plus facile, mais elle ne suit pas la qualité de la sonde. La perte n’est donc pas un bon indicateur de la qualité des représentations.

7 Discussion

Choix de conception. Le schéma retenu est le pré-norm, $x \leftarrow x + \text{MHA}(\text{LN}(x))$ puis $x \leftarrow x + \text{MLP}(\text{LN}(x))$, plutôt que le post-norm présenté en cours : il conserve un chemin résiduel direct et stabilise l’entraînement en profondeur. L’encodeur n’intègre pas les tokens masqués : il ne traite qu’un quart des tokens, ce qui réduit le calcul, et ne voit que de vrais patches, ce qui évite un écart entre pré-entraînement et déploiement [1]. La cible est normalisée par patch, ce qui améliore la qualité des représentations d’après [1] (effet non ré-ablaté ici) et impose de dénormaliser les prédictions pour afficher les reconstructions. Enfin, écrire la rétropropagation à la main sur un simple backend de tableaux oblige à comprendre chaque gradient, tout en gardant l’accélération GPU.

Limites. Le régime est très éloigné de celui de l’article d’origine : un encodeur d’un million de paramètres contre des centaines de millions, des milliers d’images contre plus d’un million, et un entraînement court. Les recons-



FIGURE 3 – STL-10. Original, image masquée (75 %) et reconstruction par le MAE.

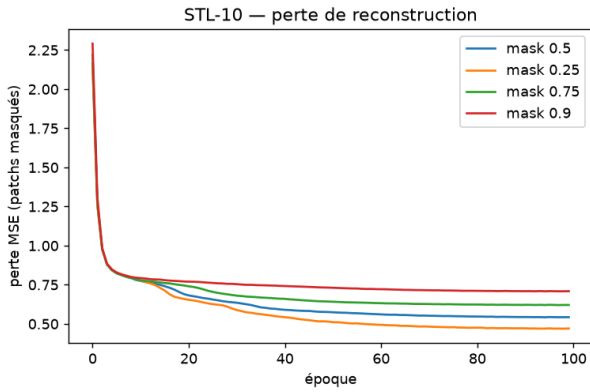


FIGURE 4 – STL-10 : perte de reconstruction par taux de masquage. Un taux plus faible donne une tâche plus facile ; les pertes ne sont pas comparables entre taux, seule la sonde mesure la qualité.

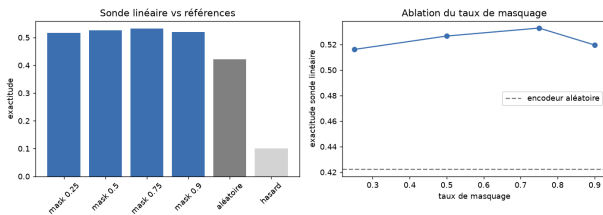


FIGURE 5 – STL-10. Exactitude de la sonde par taux de masquage, contre l’encodeur aléatoire et le hasard.

tructions, plus fines qu’avec des patches 16×16 , restent floues là où la perte quadratique pénalise peu les hautes fréquences. Les chiffres proviennent d’une seule graine. Sur NEU, les textures de défauts sont déjà assez séparables, si bien qu’un encodeur aléatoire obtient une exactitude élevée et que l’apport du pré-entraînement se lit surtout à faible nombre d’étiquettes. La rétropropagation manuelle n’est pas en cause : elle est validée par différences finies.

8 Conclusion

Un MAE a été réimplémenté de bout en bout, avec une rétropropagation entièrement manuelle et vérifiée, torch ne servant que de bibliothèque de tableaux sur GPU. Sur STL-10, le pré-entraînement dépasse un encodeur aléatoire et l’ablation du taux de masquage illustre, à petite échelle, l’étude de l’article. Sur un jeu de données industriel de défauts d’acier, le même modèle classe six types de défauts et montre l’intérêt du pré-entraînement auto-supervisé quand les étiquettes sont rares.

Références

- [1] K. He et al. *Masked Autoencoders Are Scalable Vision Learners*. CVPR, 2022.
- [2] A. Dosovitskiy et al. *An Image is Worth 16x16 Words*. ICLR, 2021.
- [3] A. Vaswani et al. *Attention Is All You Need*. NeurIPS, 2017.
- [4] J. Devlin et al. *BERT*. NAACL, 2019.
- [5] H. Bao, L. Dong, F. Wei. *BEiT*. arXiv :2106.08254, 2021.
- [6] P. Vincent et al. *Stacked Denoising Autoencoders*. JMLR, 2010.
- [7] D. Kingma, J. Ba. *Adam*. ICLR, 2015.
- [8] I. Loshchilov, F. Hutter. *Decoupled Weight Decay Regularization*. ICLR, 2019.
- [9] K. Song, Y. Yan. *A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects*. Applied Surface Science, 2013.
- [10] Y. He et al. *An end-to-end steel surface defect detection approach*. IEEE TIM, 2020.
- [11] P. M. Bhatt et al. *Image-Based Surface Defect Detection Using Deep Learning : A Review*. ASME JCISE, 2021.
- [12] B. Tang et al. *Review of surface defect detection of steel products*. IET Image Processing, 2023.